

BÁO CÁO BÀI TẬP VỀ NHÀ

Nội dung: Week 2 - Session 3 (Introduction to DSA)

Người thực hiện: Lê Phạm Mỹ Ngọc

I. Viết lại thuật toán Binary Search mà không cần nhìn ghi chú

Đoạn mã Javascript:

```
JS BinerySearchPractice.js > ...
1  let arr = [2, 3, 9, 17, 28, 39, 57, 71, 89, 90, 110, 221, 334],
2      arrayLength = arr.length;
3  let left = 0,
4      right = arrayLength - 1,
5      mid = 0;
6  let numberNeedToBeFound = 39,
7      result = -1;
8  while (left <= right) {
9      mid = Math.floor(left + (right - left) / 2);
10     if (arr[mid] === numberNeedToBeFound) {
11         result = mid;
12         break;
13     }
14     if (arr[mid] < numberNeedToBeFound) left = mid + 1;
15     else right = mid - 1;
16 }
17 if (result !== -1) console.log("Index của số cần tìm là :", result);
18 else console.log("Không tìm thấy số cần tìm trong mảng.");
19
```

II. Tìm hiểu Bubble Sort

1. Nguyên lý hoạt động

Thuật toán này hoạt động bằng cách lặp đi lặp lại việc hoán đổi hai phần tử kế tiếp nhau nếu chúng đang đứng sai thứ tự (phần tử đứng trước lớn hơn phần tử đứng sau trong trường hợp sắp xếp tăng dần).

2. Các bước thực hiện:

1. So sánh cặp phần tử đầu tiên. Nếu phần tử thứ nhất $>$ phần tử thứ hai, hoán đổi chúng. (Trong trường hợp xếp thứ tự tăng dần)
2. Tiếp tục so sánh cặp tiếp theo (thứ hai và thứ ba) và hoán đổi nếu cần.
3. Lặp lại cho đến cuối mảng. Sau lượt duyệt đầu tiên, phần tử lớn nhất chắc chắn sẽ nằm ở vị trí cuối cùng.
4. Lặp lại quá trình trên với các phần tử còn lại (loại trừ phần tử cuối đã nằm đúng chỗ).

3. Ví dụ

Mảng ban đầu: [3, 7, 4, 0]

- **Duyệt lần 1:**

- So sánh 3 và 7: $3 < 7 \rightarrow$ Giữ nguyên $\rightarrow$ [3, 7, 4, 0]
- So sánh 7 và 4: $7 > 4 \rightarrow$ Đổi chỗ 7 và 4 $\rightarrow$ [3, 4, 7, 0]
- So sánh 7 và 0: $7 > 0 \rightarrow$ Đổi chỗ 7 và 0 $\rightarrow$ [3, 4, 0, 7]

$\rightarrow$ Lúc này số 7 (Giá trị lớn nhất) đã nổi về cuối dãy.

- **Duyệt lần 2:**

- So sánh 3 và 7: $3 < 7 \rightarrow$ Giữ nguyên $\rightarrow$ [3, 4, 0, 7]
- So sánh 4 và 0: $4 > 0 \rightarrow$ Đổi chỗ 4 và 0 $\rightarrow$ [3, 0, 4, 7]
- Không xét 4 so với 7 nữa vì 7 chắc chắn đã nằm đúng chỗ là giá trị lớn nhất.

$\rightarrow$ Lúc này số 4 (Giá trị lớn thứ 2) đã nổi về phía cuối dãy, đứng trước giá trị lớn nhất.

- **Duyệt lần 3:**

- So sánh 3 và 0: $3 > 0 \rightarrow$ Đổi chỗ 3 và 0 $\rightarrow$ [0, 3, 4, 7]
- 4 và 7 đã nằm đúng chỗ nên ngừng so sánh.

$\rightarrow$ Lúc này số 3 (Giá trị lớn thứ 3) đã nổi về hướng cuối dãy, đứng trước 2 giá trị lớn hơn nó.

- **Kết quả:** [0, 3, 4, 7]

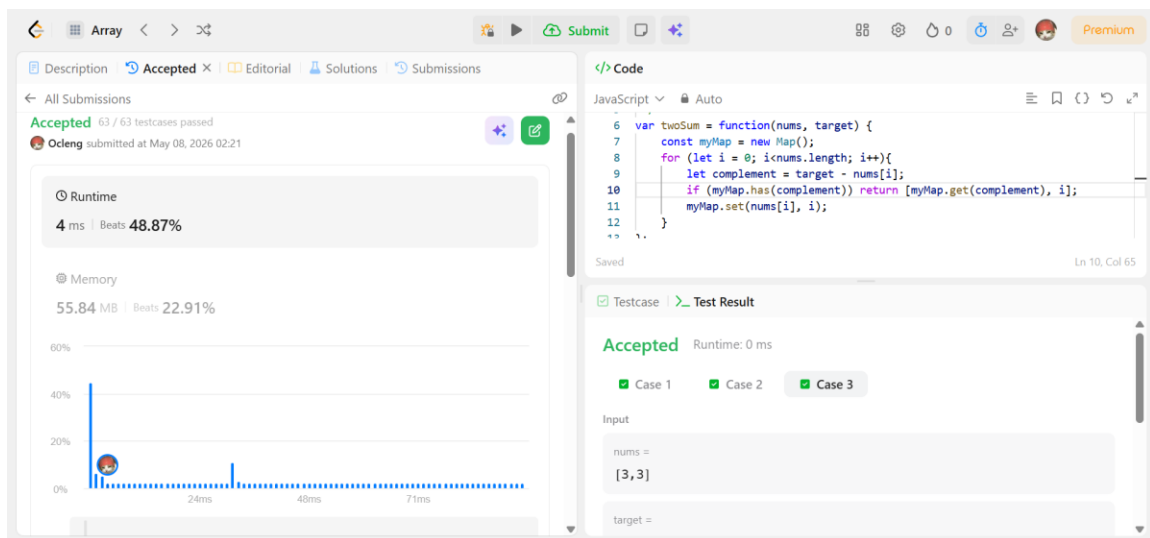
4. Độ phức tạp Big-O

- **Thời gian (Time Complexity):**
 - **Trường hợp tệ nhất: $O(n^2)$** - Khi mảng bị ngược hoàn toàn. Phải dùng 2 vòng lặp lồng nhau, mỗi vòng chạy n lần.
 - **Trường hợp tốt nhất: $O(n)$** - Khi mảng đã được sắp xếp sẵn (nếu có thêm biến kiểm tra để thoát sớm, ví dụ như một biến đếm count để đếm số phần tử nằm đúng vị trí, nếu đếm được tất cả phần tử đều nằm đúng thì thoát).
- **Không gian (Space Complexity): $O(1)$** - Vì chỉ tốn một biến tạm để hoán đổi, không cần thêm mảng phụ.

III. Giải 3 câu leetcode mức độ Easy về Array & Chỉ ra độ phức tạp Big-O của thuật toán đã dùng

5. Two Sum

Vì dùng 2 vòng lặp for lồng vào nhau để xét tất cả trường hợp thì sẽ đây Big-O thành $O(n^2)$, em áp dụng kỹ thuật Hash Map để giảm thời gian truy xuất về $O(1)$. Nhưng để thực hiện được thì cần tốn $O(n)$ về bộ nhớ để lưu lại mảng dưới dạng có thể truy xuất từ giá trị. Cụ thể, em duyệt mảng một lần và lưu trữ các giá trị đã gặp vào Map dưới dạng cặp Key là giá trị phần tử và Value là chỉ số của nó. Tại mỗi bước, em kiểm tra xem giá trị hiệu số giữa target và phần tử hiện tại có tồn tại trong Map không. Tổng thể thuật toán chỉ tốn $O(n)$ về thời gian xử lý và $O(n)$ về bộ nhớ để lưu các cặp giá trị, việc truy cập theo key chỉ tốn $O(1)$ nên độ phức tạp cuối cùng là $O(n)$.

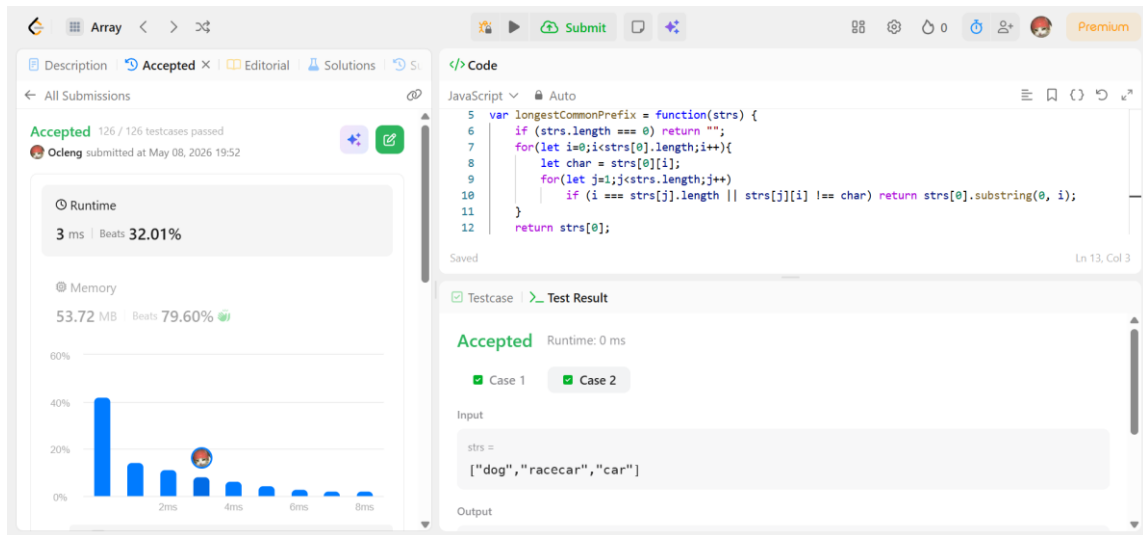


Big-O: $O(n)$

6. Longest common prefix

Thay vì so sánh từng cặp chuỗi một cách tuần tự, em áp dụng kỹ thuật quét theo chiều dọc. Điểm cốt lõi của phương pháp này là lấy từng ký tự của chuỗi đầu tiên làm mốc và so khớp theo từng 'cột' với tất cả các chuỗi còn lại. Về độ phức tạp, dù sử dụng hai vòng lặp lồng nhau, nhưng thuật toán không vận hành theo quy luật $O(n^2)$ (trong đó n là số lượng chuỗi). Thay vào đó, độ phức tạp thời gian chính xác là $O(n*m)$ (với n là số lượng chuỗi, m là độ dài của chuỗi ngắn nhất) vì mỗi ký tự chỉ được so sánh với các ký tự cùng vị trí. Điều này có nghĩa là thời gian xử lý tỉ lệ

thuận với tổng lượng dữ liệu thực tế, thay vì tăng theo bình phương số lượng chuỗi. Em chọn thuật toán này bởi vì nghĩ đến những trường hợp mà những phần tử đầu trùng nhau rất nhiều kí tự nhưng ở các phần tử sau lại chỉ có ít hoặc không có kí tự trùng, điều này khiến cho thời gian xét trùng nhiều kí tự của những phần tử đầu trở nên vô nghĩa. Thế nên, thay vào đó mà dùng thuật toán quét dọc này sẽ đảm bảo chỉ cần phát hiện một ký tự khác biệt hoặc một chuỗi chạm giới hạn độ dài, thuật toán sẽ thực hiện Early Exit (thoát sớm) và trả về kết quả ngay lập tức bằng phương thức substring.

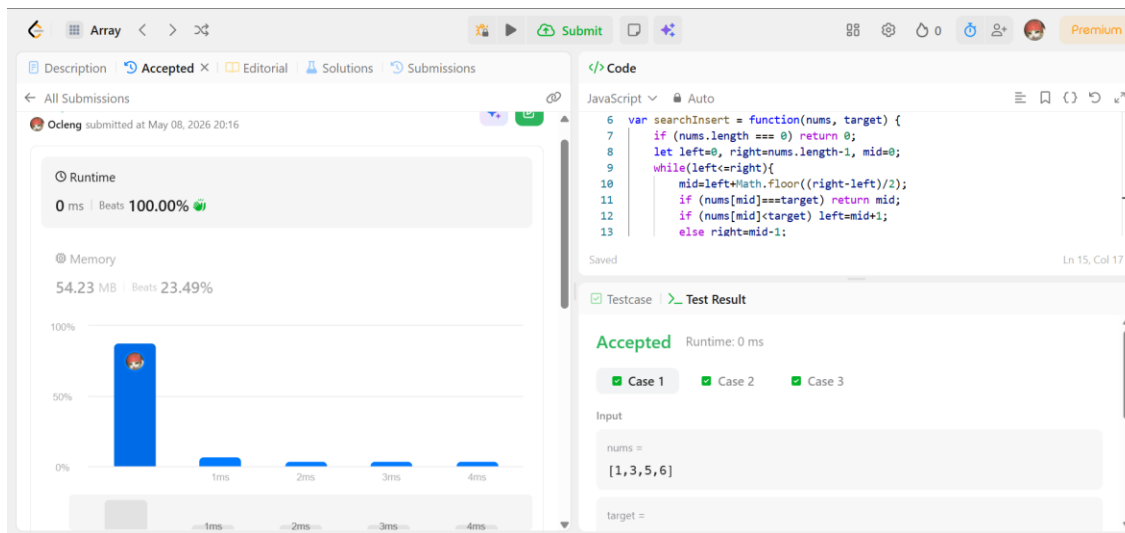


Big-O: $O(n^*m)$

7. Search insert position

Ở bài toán này, có thể dễ dàng thấy đây là một bài toán tìm kiếm. Đề bài đã nêu rõ rằng dãy số đã được sắp xếp và yêu cầu viết theo độ phức tạp $O(\log n)$, thế nên em viết theo thuật toán Binary Search (Tìm kiếm nhị phân). Thuật toán tính mốc giữa dựa theo mốc trái và phải (ban đầu chính là phần tử đầu tiên và cuối cùng của dãy số) để so sánh với giá trị mục tiêu, nếu giá trị mục tiêu bằng mốc giữa thì trả về vị trí mốc giữa, nếu lớn hơn mốc giữa thì tiếp tục xét dãy số bên phải (cập nhật mốc trái thành mốc giữa +1, sau đó cập nhật mốc giữa và xét lại), và nếu nhỏ hơn thì tiếp tục xét dãy số bên trái (cập nhật mốc phải thành mốc giữa -1, sau đó cập nhật mốc giữa và xét lại). Lặp đi lặp lại cho đến khi mốc trái và mốc phải giao nhau. Do đề

bài yêu cầu tìm vị trí chèn nên sẽ không trả về -1 nếu không tìm thấy, mà trả về giá trị mốc trái.



Big-O: $O(\log n)$

--Hết--